

1. Linear Program:

- ① Objective function + Constraints are all linear
- ② Typically (Not always) variables are non-negative
- ③ Uses only non-strict inequality ($\leq, \geq$)

2. Function can be written as $f: \mathbb{R}^n \rightarrow \mathbb{R}$, $f(x) = a^T x + b$,
is an affine function.

3. Linear function: $f(x) = Ax$ (Pass through the origin)
Affine function: $f(x) = Ax + b$

4. Affine function is more general, linear function is a special case of affine function. ($b = 0$)

5. Hyperplane: The region defined by a linear equality $a_i x = b_i$
Halfspace: The region defined by a linear inequality $a_i x \leq b_i$

6. Feasible region: The set of all points that satisfies all the constraints of a LP problem.

7. Fundamental theorem of LP: If a linear program has an optimal solution, then there exists at least one optimal solution at a vertex (corner point) of a feasible region.

8. An unbounded feasible region doesn't mean the LP is unbounded.

9. Optimal solution: The value of the decision variables that gives the best result for the objective function.

10. Optimal objective value: The best value achieved by the objective function.

Summary of graphical method

- Works best for LPs with **two variables**.

Steps:

- Plot all constraints to identify the **feasible region**.
- Draw **objective function level lines** (parallel lines for different values).
- Move the level lines in the direction of optimization (maximize/minimize).
- **Optimal solution** occurs at a **vertex** (corner point) of the feasible region.

Limitations: Works best for 2 variables (easy to draw). For higher dimensions, use **Simplex or other algorithms**.

Three types of LPs

Every LP falls into one of these cases:

Infeasible — No solution satisfies all constraints.

Unbounded — Objective can improve indefinitely within the feasible region.

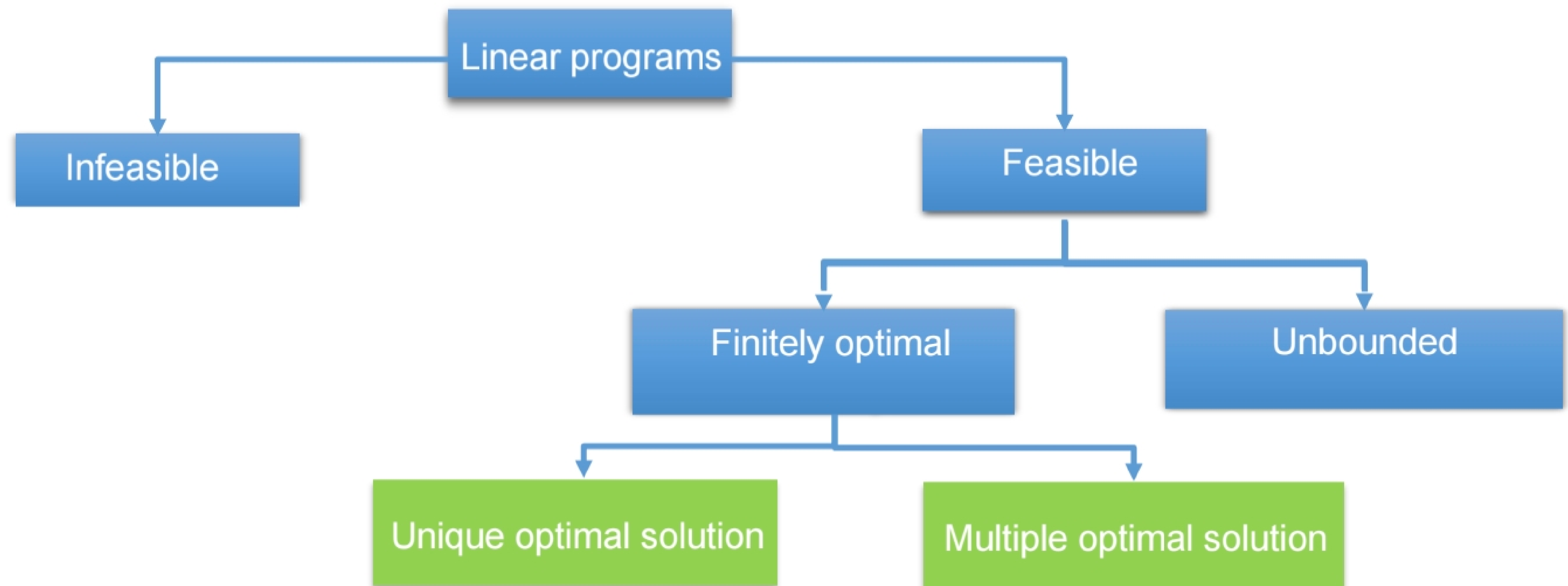
Finitely optimal — An optimal solution exists and is finite.

If finitely optimal, the solution can be:

Unique, or

Multiple optimal solutions along an edge or face of the feasible region.

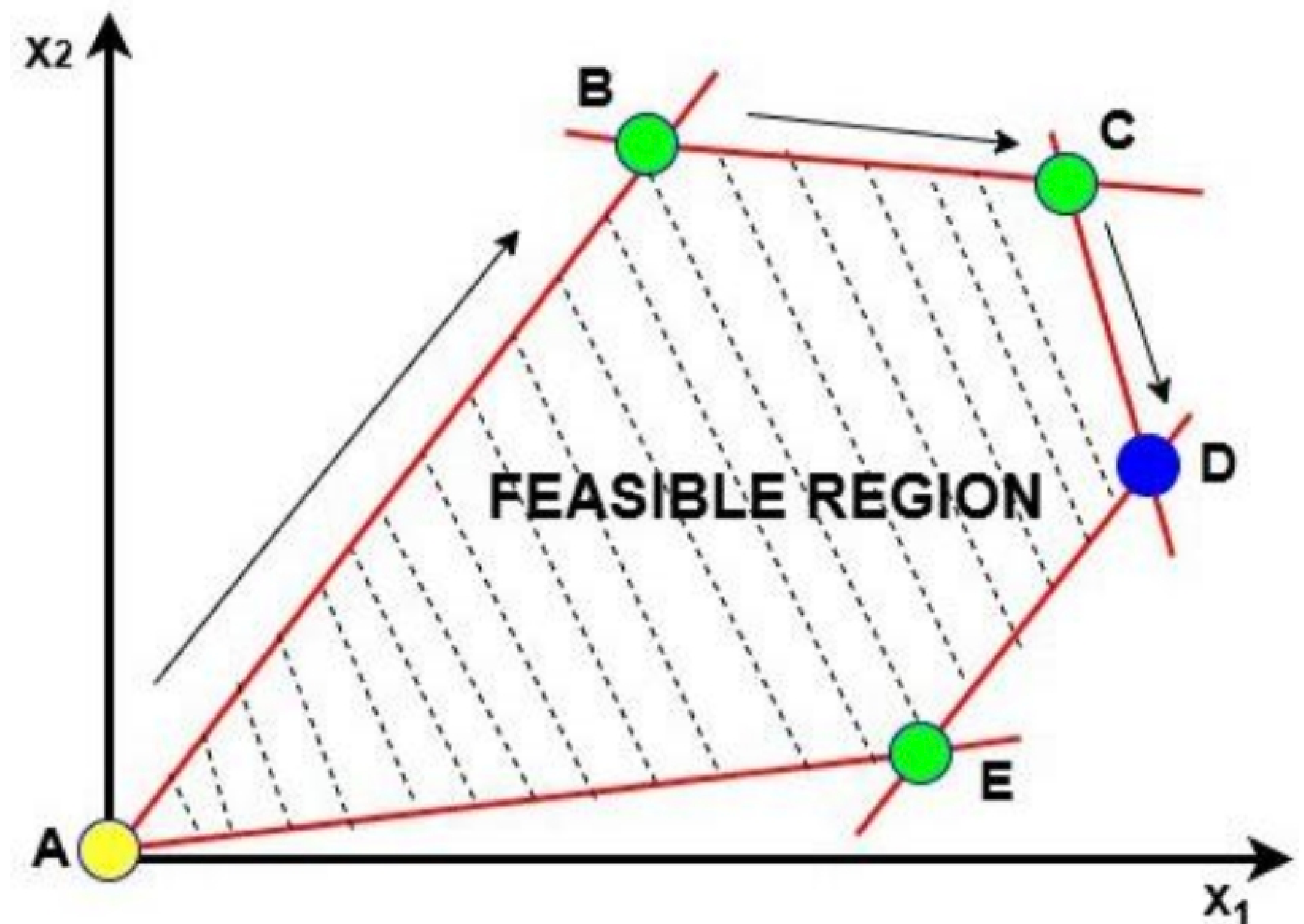
Summary: Different cases of LP outcomes



Simplex algorithm

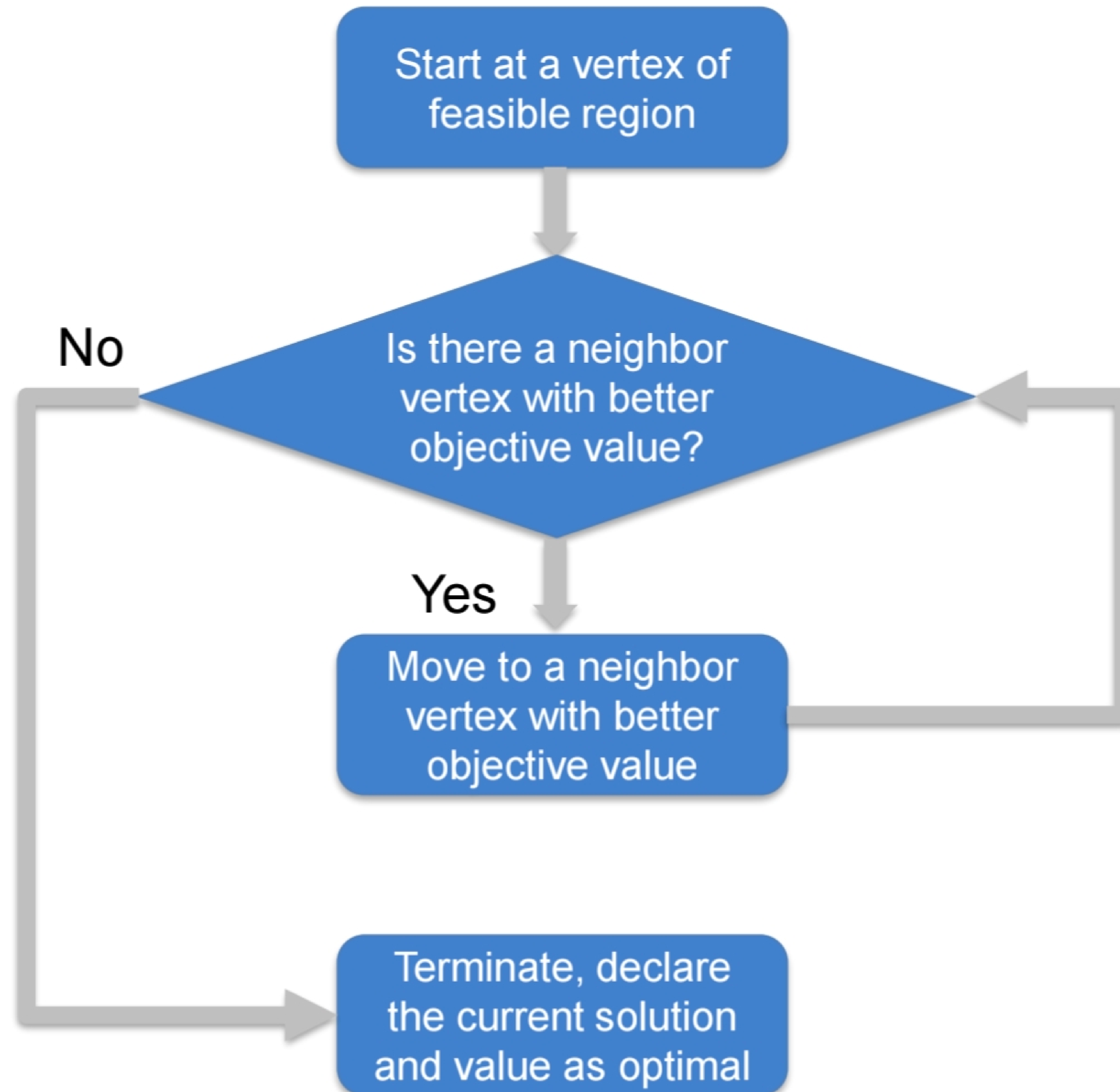
Basic idea:

Move from vertex to vertex along the edges of the feasible polytope, always in a direction that improves the objective function, until no further improvement is possible.

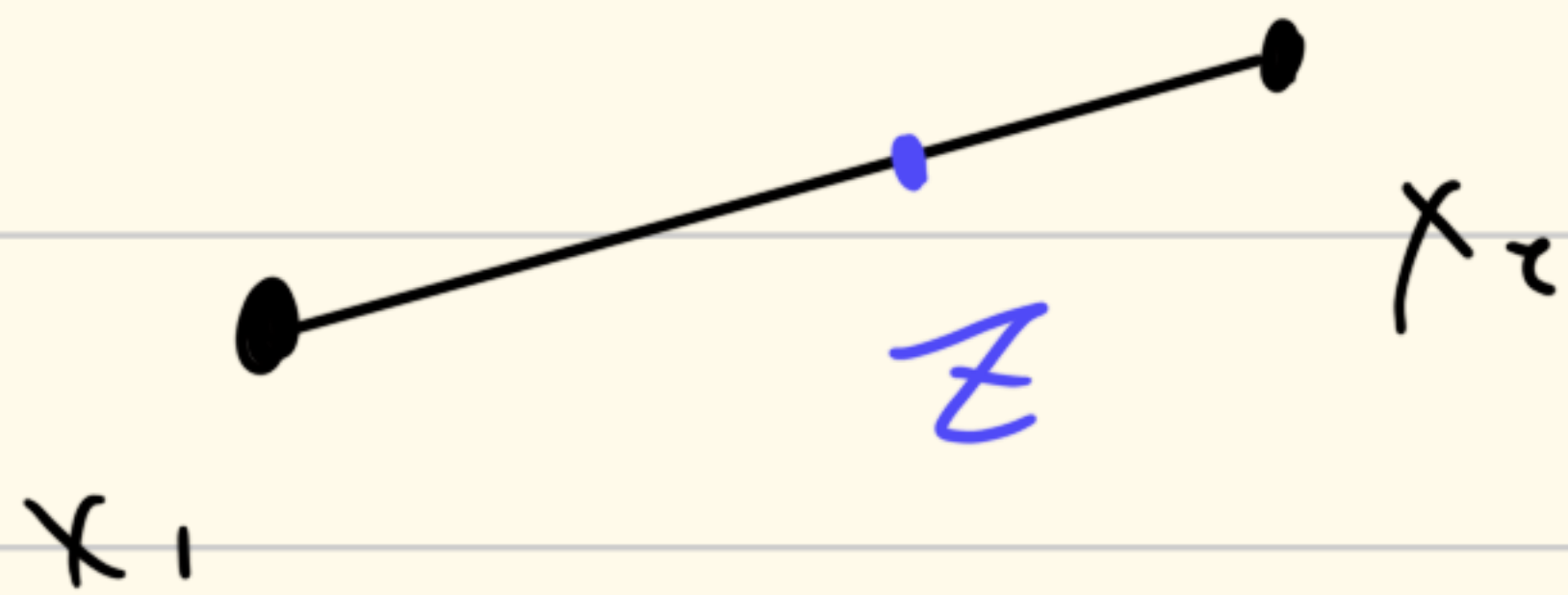


Simplex = *systematic exploration* of corners of the feasible region.
Relies on the same idea: optimal solutions lie on vertices.

Simplex algorithm implementation



1. Given $x_1, x_2 \in \mathbb{R}^n$, a convex combination of them is any points of the form $z = \theta x_1 + (1-\theta)x_2$ where $\theta \in [0, 1]$.
- When $\theta \in (0, 1)$, z is called a strict convex combination of x_1, x_2 .
 - A point between 2 points.



$z \cdot \nabla_x f(x)$ points in the direction where f increases most rapidly. So $-\nabla_x f(x)$ is the direction of steepest descent.

3. Gradient descent takes larger steps when slope is steep. As slope goes flatten, gradient descent takes smaller step to converge to local minimum.

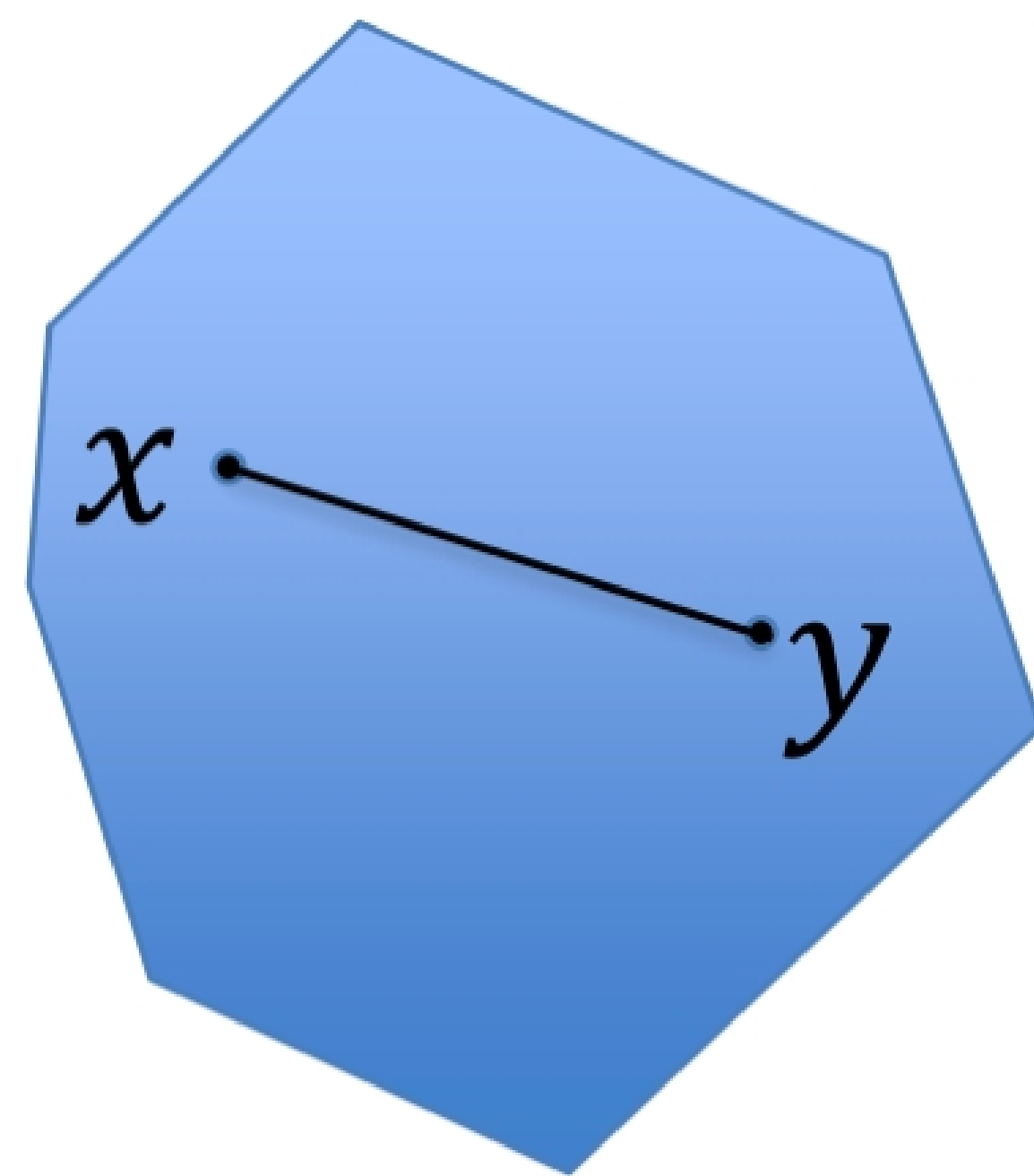
4. Gradient descent works for functions that aren't strictly convex.

Convex set: Definition

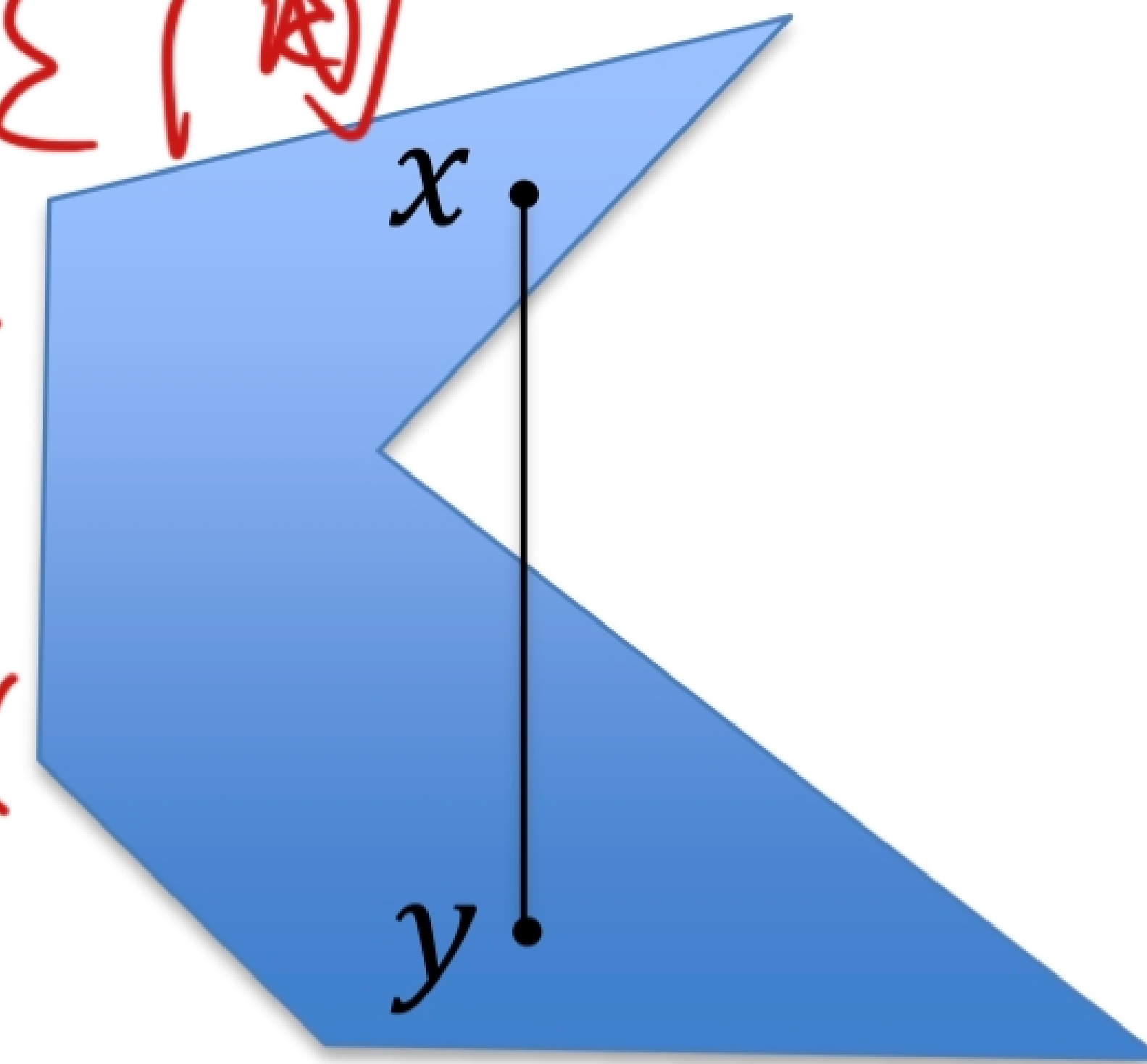
A convex set is a set $C \subseteq R^n$ such that for all $x_1, x_2 \in C$ and $\theta \in [0, 1]$ we have :

$$\theta x_1 + (1 - \theta)x_2 \in C$$

任何可在 x, y 之间的
 点都在 set
 之中 $\rightarrow$ convex
 set



Convex set

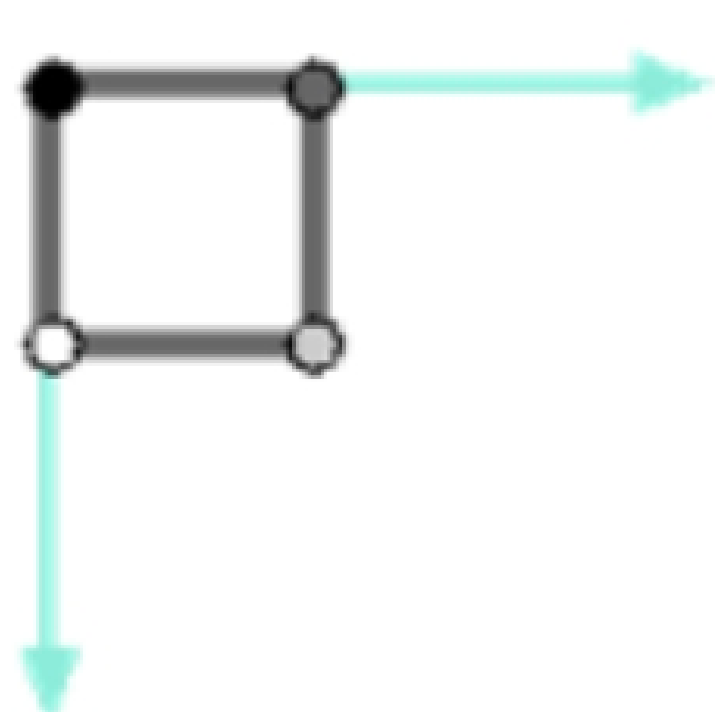


Non-convex set

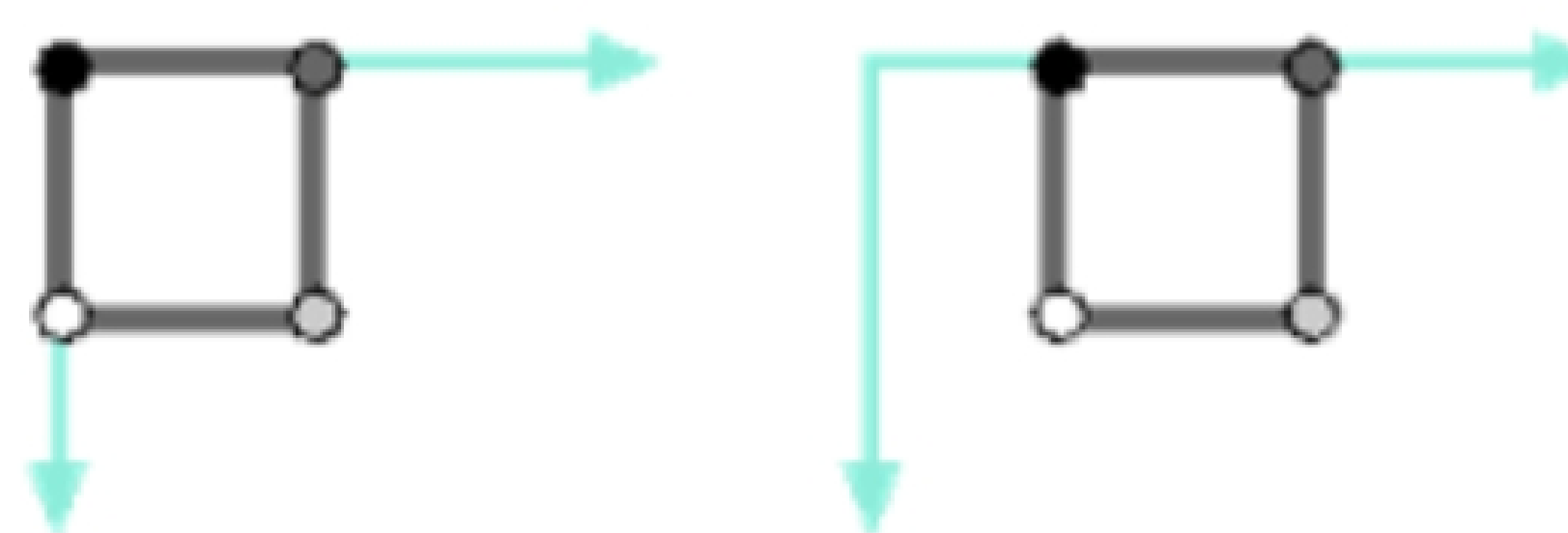
Convex set: Properties

- The empty set $\emptyset$ and $\mathbb{R}^n$ are both convex.
- Intersections of convex sets are convex.
- Convexity is **preserved** under three common operations:
 - **Scaling:** Multiplying all points by a positive scalar
 - **Translation:** Shifting to a new position (adding a constant vector to all points)
 - **Rotation:** Rotating them around an axis

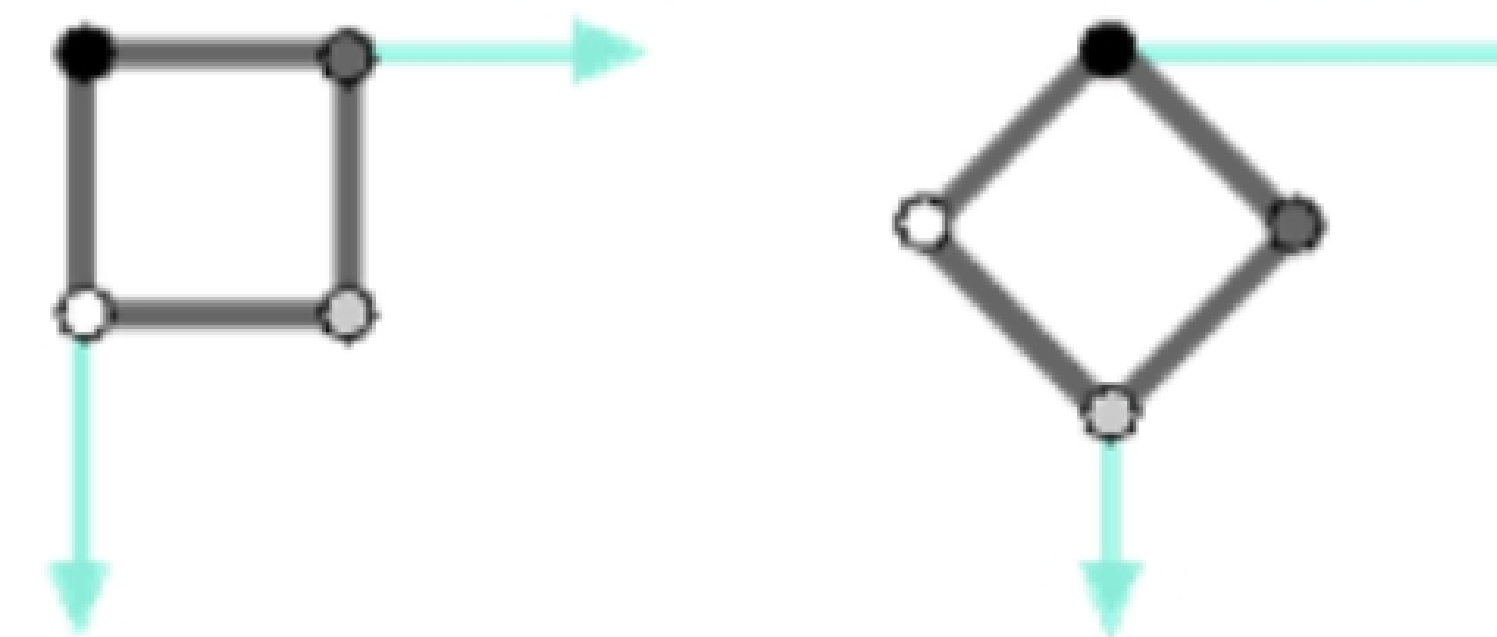
Scale



Translation



Rotation



Convex function: Definition

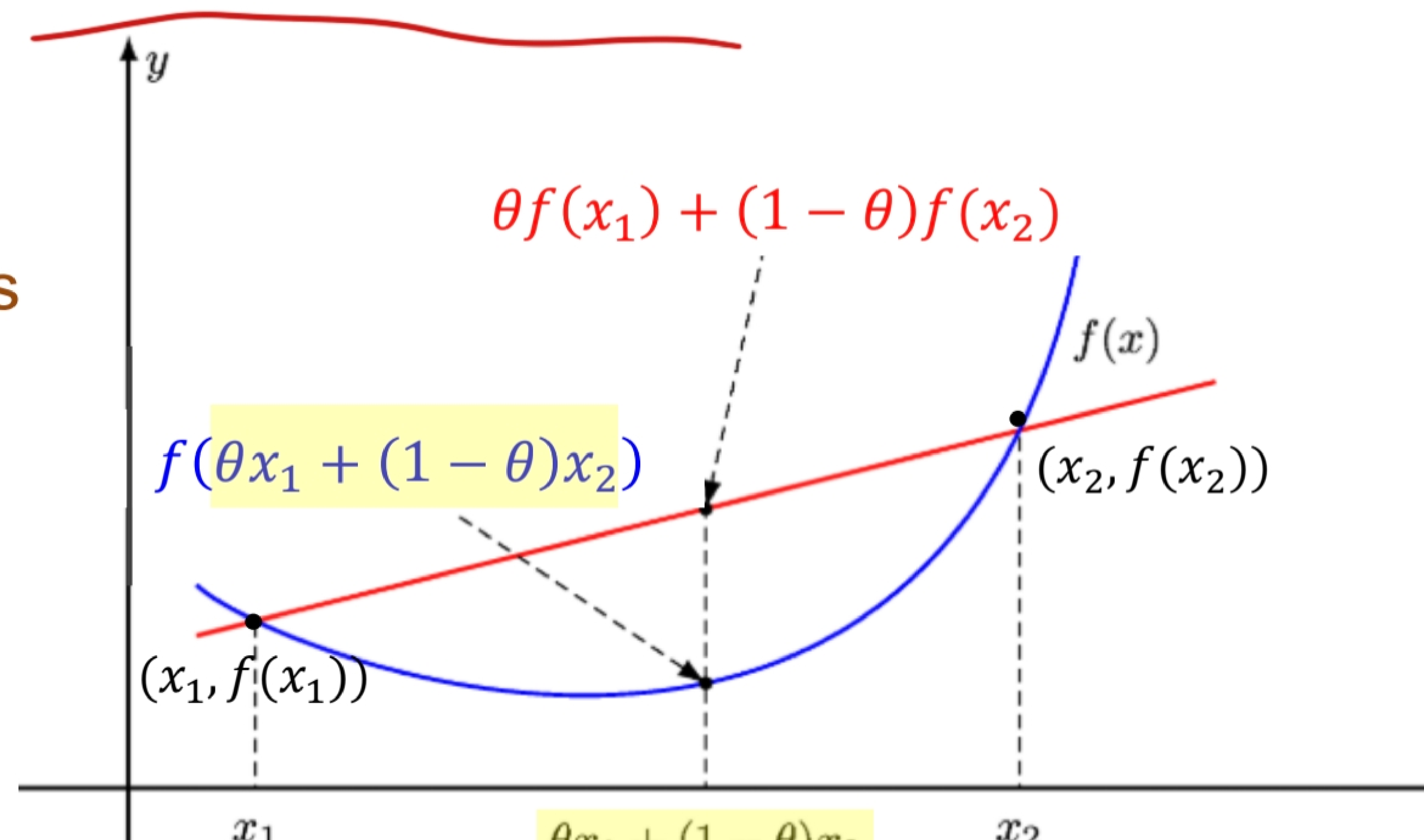
Let C be a convex set. A function $f: C \rightarrow \mathbb{R}$ is convex in C if $\forall x_1, x_2 \in C, \forall \theta \in [0,1]$,

$$f(\theta x_1 + (1 - \theta)x_2) \leq \theta f(x_1) + (1 - \theta)f(x_2)$$

If $C = \mathbb{R}^n$, we simply say f is a convex function.

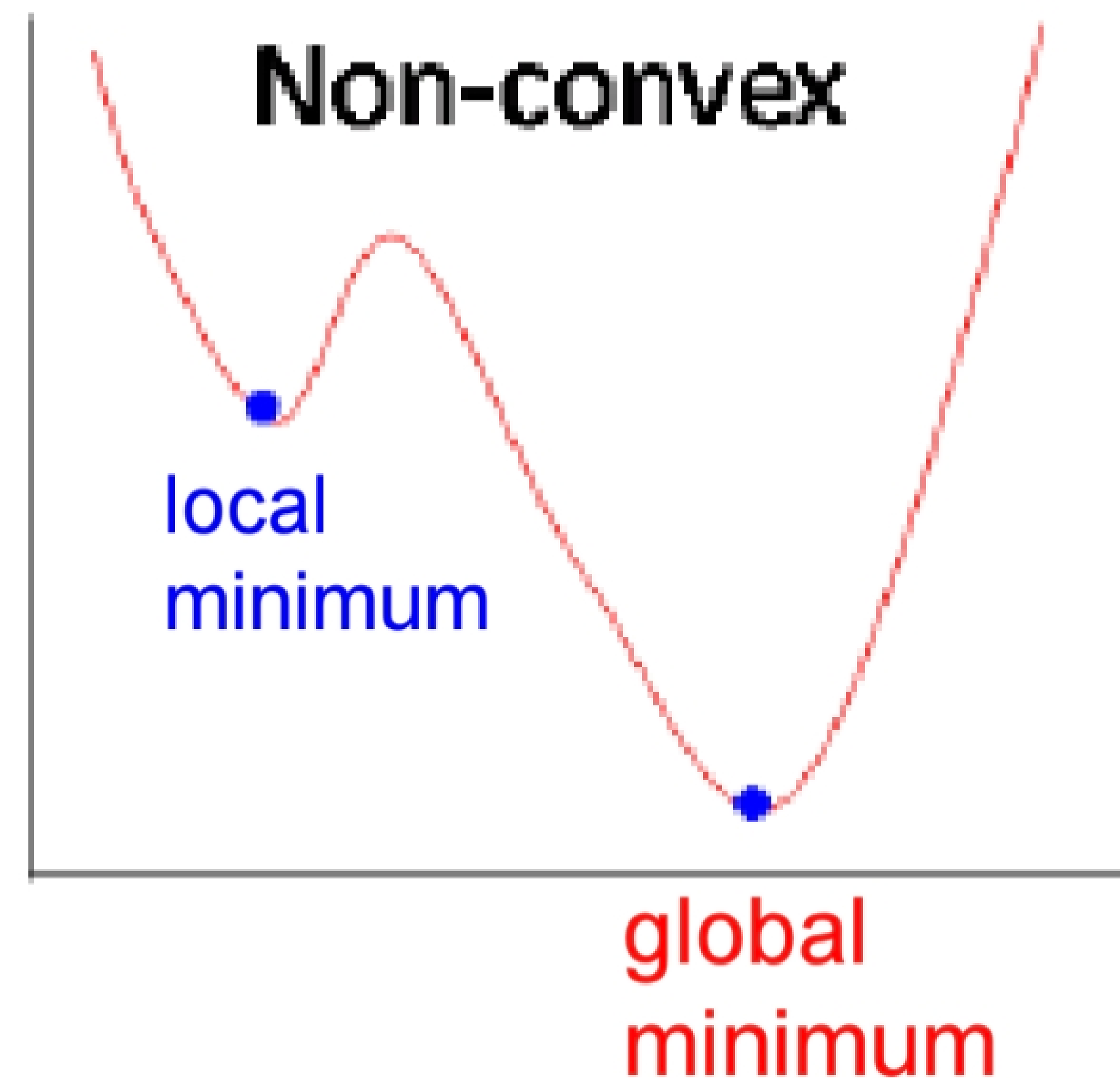
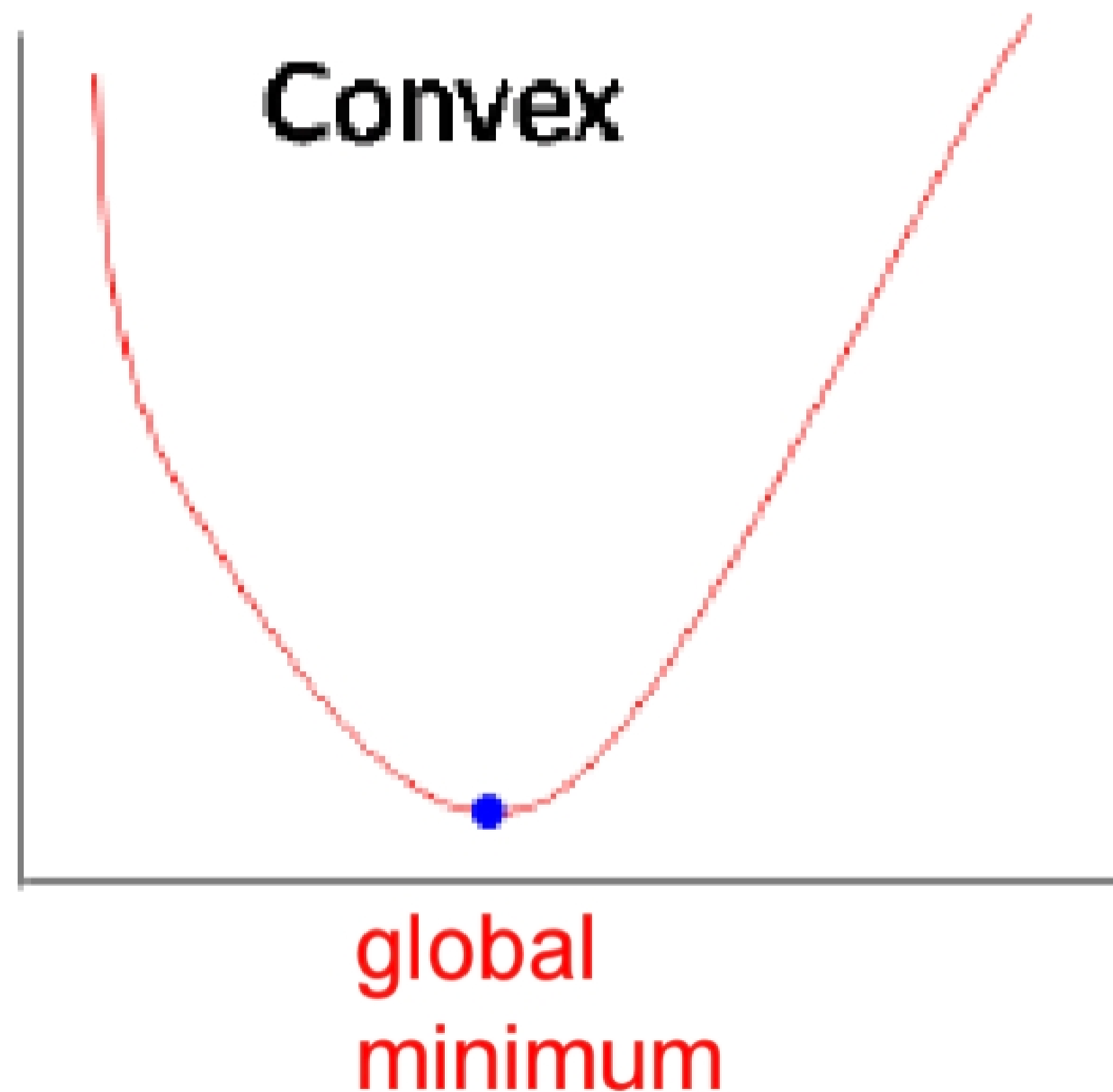
Any chord between two points on the function always lies above the function.

Value in the middle point is lower than the average of the two endpoint values.



Convex function: Properties

- Sum of convex functions is convex
 - $af(x) + bg(x)$ is convex for convex functions f, g and $a, b > 0$.
- Convexity is preserved under affine transformation
 - If $f(\mathbf{x}) = g(\mathbf{Ax} + \mathbf{b})$, g is convex, then $f(\mathbf{x})$ is convex.
where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$
- Any **local minimum** is a **global minimum**



Gradient descent algorithm

Key points:

- $-\eta \nabla_x f(\mathbf{x}_k)$: The product of the *learning rate* and the *gradient*, which determines the *direction* and *step size* to move towards minimizing the cost.
- Common stopping criteria:
 - Maximum number of iterations is reached.
 - The percentage or absolute change in f is below a threshold.
 - The percentage or absolute change in the variable $\mathbf{x}$ is below a threshold.

Gradient Descent:

Initialize $\mathbf{x}_0$ and learning rate η

while true **do**

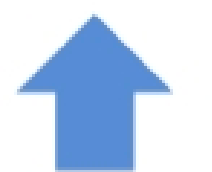
 Compute $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \eta \nabla_x f(\mathbf{x}_k)$

if converge **then**

return $\mathbf{x}_{k+1}$

end

end



Gradient descent algorithm

Choose suitable learning rate:

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k - \eta \nabla_{\mathbf{w}} C(\mathbf{w}_k)$$

If η is too small...

Gradient descent may be slow.

If η is too large...

Gradient descent may:

- Overshoot, never reach minimum
- Fail to converge, diverge

